1 Paper review

Sensor failures and transmission losses leave gaps in multivariate time series. Interpolation, neighbouring-sensor averages and matrix factorization estimate these gaps from temporal patterns or similarities between channels. Recurrent methods such as BRITS also capture cross-channel dependencies, but do not explicitly encode a sensor network. GRIN introduces this structure so that reconstruction can use nonlinear relationships between connected sensors as well as their history *[1, Sections 1–2].*

Preliminaries and the GRIN idea

At time t, each of N identified sensors supplies d measurements, stored in $X_t \in \mathbb{R}^{Nxd}$. The fixed weighted adjacency matrix W describes sensor relationships. The binary mask M_t has the same shape as X_t: 1 means observed and 0 means missing. GRIN predicts gaps from previous memory, refines them through connected sensors and updates that memory. A reverse-time module repeats this process, allowing observations on both sides of a gap to contribute see Figure 1. *[1, Sections 3–4].*

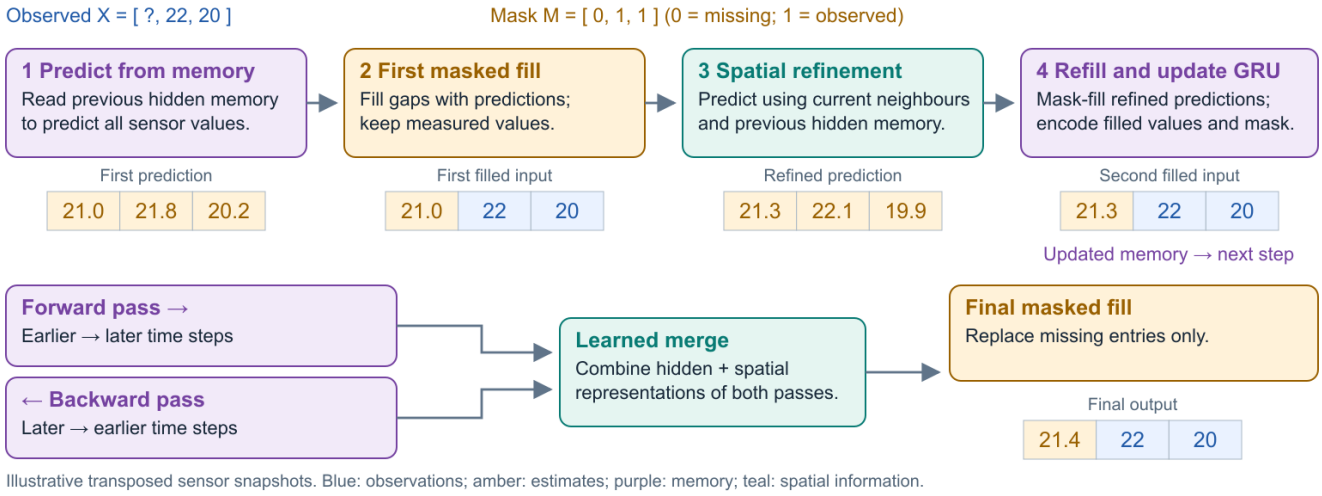


Figure 1 GRIN processing steps with invented values. Rows list sensors A–C horizontally: for d = 1 these are transposed X_t snapshots. The mask remains unchanged; observed values are preserved.

Temporal prediction and selective filling

The first readout predicts the current measurements from previous memory H_{t-1}. This memory summarises earlier readings and their spatial relationships. The filler Φ keeps entries with mask value 1 and inserts predictions where the mask is 0. The unchanged mask accompanies the filled data so that estimated readings remain distinguishable from observations *[1, Eqs. 7–8].*

$$\hat{Y}_t^{(1)} = H_{t-1}V_h + b_h; \quad \Phi(Y_t) = \underbrace{M_t \odot X_t}_{\text{keep observed}} + \underbrace{(1 - M_t) \odot Y_t}_{\text{fill missing}}$$

Spatial refinement through message passing

Neighbours send learned messages containing their filled readings, masks and previous memories. Their messages are summed and combined with the target sensor's previous memory to form S_t. The target's current reading is excluded from its own spatial reconstruction, preventing direct copying. The second readout converts S_t and previous memory into refined predictions; applying Φ again preserves observed measurements *[1, Eqs. 9–10].*

$$\hat{Y}_t^{(2)} = \underbrace{[S_t \| H_{t-1}]}_{\text{neighbours and prior memory}} V_s + b_s; \quad \hat{X}_t^{(2)} = \underbrace{\Phi(\hat{Y}_t^{(2)})}_{\text{fill only missing entries}}$$

The spatial representation aggregates information from neighbouring sensors; the readout maps this information and previous memory back to measurement values.

Memory update

A message-passing GRU then reads the refined, filled measurements, their masks and previous memory. Its reset gate determines how past memory enters the candidate c_t; its update gate u_t balances the candidate against retained memory. Graph-based gates also account for neighbouring sensors. The new H_t supports prediction at the next time step *[1, Eqs. 3–6]*.

$$h_t^i = \underbrace{u_t^i \odot h_{t-1}^i}_{\text{retained memory}} + \underbrace{(1 - u_t^i) \odot c_t^i}_{\text{candidate contribution}}$$

Bidirectional reconstruction

The backward module applies the same steps in reverse chronological order, so its memory contains information from later times. An MLP learns how to combine spatial representations and memories from both directions. It combines learned features rather than simply averaging two predictions. Applying Φ to this final output preserves all observed values *[1, Eq. 11]*.

$$\hat{y}_t^i = MLP([\ \underbrace{s_t^{i,fwd} \| h_{t-1}^{i,fwd}}_{\text{forward information}} \ \| \ \underbrace{s_t^{i,bwd} \| h_{t+1}^{i,bwd}}_{\text{backward information}}\ ])$$

In short: GRIN estimates gaps from temporal memory, improves these estimates using related sensors, and carries the resulting information through the sequence. Combining forward and backward representations uses context on both sides of a gap, while the mask preserves observed measurements.

Evaluation and reported performance

GRIN is compared with statistical and neural baselines using MAE, MSE and mean relative error, averaged over five runs. Evaluation targets are withheld from inputs. Air-quality tests assess in-sample and out-of-sample reconstruction; traffic and energy tests cover 25% random point losses and blocks combining random omissions with longer outages. Training supervises final and intermediate predictions in both directions *[1, Section 5; Appendix A].*

Table 1 Published out-of-sample MAE against the best competing baseline in each setting; lower is better. Reduction = 100 × (baseline – GRIN) / baseline. Means from [1, Tables 1–2]; uncertainty omitted. MAE units differ between datasets, so comparisons apply within each row

Dataset and setting	Best other model	Other MAE	GRIN MAE	Reduction
AQI-36 · air quality	BRITS	14.50	12.08	16.7%
AQI · air quality	MPGRU	18.76	14.73	21.5%
PEMS-BAY · blocks	MPGRU	1.59	1.14	28.3%
PEMS-BAY · points	MPGRU	1.11	0.67	39.6%
METR-LA · blocks	BRITS	2.34	2.03	13.2%
METR-LA · points	BRITS	2.34	1.91	18.4%
CER-E · blocks	MPGRU	0.53	0.42	20.8%
CER-E · points	MPGRU	0.41	0.29	29.3%

GRIN achieves the lowest reported MAE in these settings. Ablations support spatial decoding and bidirectionality, especially for blocks. Diffusion convolution limits propagation to local neighbourhoods. Virtual sensing extends reconstruction to entirely unobserved nodes when connected sensors provide sufficient information *[1, Sections 5.1–5.2; Appendix A].*

Conclusion

GRIN uses sensor relationships to improve imputation across the evaluated domains. It depends on useful relational information and representative training data and largely assumes stationarity. Bidirectionality requires later observations, limiting immediate online use. Reconstruction guarantees and adaptation to changing conditions remain open *[1, Section 6].*

2 Reproducing the results

2.1 The original code

The AQI-36 out-of-sample experiment was selected because it is a main benchmark in the paper and can be evaluated using the authors' fixed missing-value mask. The original GRIN model was downloaded from commit 4a28afbb092600b6e6abeeabaaf67e87dbd1ed6e and verified using SHA-256. A small PyTorch training harness replaced the original Lightning runner to support Python 3.12. The architecture was retained; no claim of an independent reimplementation is made.

2.2 Code implementation

The Jupyter notebook downloads the versioned dataset, creates the graph, trains the model and evaluates the selected checkpoint. The released data contain 8,759 hourly observations at 36 stations. March, June, September and December are reserved for testing. Validation windows precede these months; overlapping training windows are removed. The resulting 4,948 training, 564 validation and 2,788 test windows share no observations. Standardisation is fitted on visible training data. Evaluation targets remain hidden during validation and testing.

Execution parameters

Parameter group	Values used
Data and windows	AQI-36; 36 hours; stride 1; validation fraction 0.10; stored evaluation mask
Graph	Geographic Gaussian kernel; threshold 0.10; no self-loops; 654 directed entries
Architecture	Hidden 64; feed-forward 64; node embedding 8; one recurrent layer; MLP merge
Graph operations	Encoder diffusion order 2; decoder order 1; global attention off; no exogenous inputs
Regularisation	Dropout 0; feed-forward dropout 0; layer normalisation off; weight decay 0
Training budget	30 epochs; 512 windows per epoch sampled with replacement; batch size 32
Optimisation	Adam; learning rate 0.001; cosine decay to 0.0001 over 30 epochs; gradient norm clip 5
Loss and masking	Sum of five scaled masked MAEs; intermediate loss weight 1; additional masking 0.05
Selection and randomness	Minimum validation window MAE; patience 40; seed 42; deterministic PyTorch operations
Runtime	CPU; one PyTorch thread; Python 3.12.14; PyTorch 2.14.1+cpu; exact dependencies in repository
Final scoring	Average overlapping test predictions by timestamp, then score the fixed held-out targets

The paper configuration permits 300 epochs and 5,120 windows per epoch, with five runs. The local budget was reduced for CPU execution; the cosine schedule was shortened accordingly. The notebook also provides the paper-budget configuration.

2.3 Comparison to the paper's result

The run completed 30 epochs and 480 optimiser updates in 13.3 minutes. The checkpoint from epoch 18 was selected using validation data and then scored on 20,434 withheld test readings. All errors below use the original PM2.5 scale; lower is better. Published results are five-run means, while local results come from one seed.

Method	Result source	MAE	MSE
GRIN	Local reduced-budget run	12.789	501.39
GRIN	Paper, out-of-sample	12.08 ± 0.47	523.14 ± 57.17
BRITS	Paper, out-of-sample	14.50 ± 0.35	662.36 ± 65.16
Station mean	Local training-only control	55.184	4721.55
Linear interpolation	Local bidirectional control	14.598	691.16

The local GRIN MAE differs from the published mean by +0.709 $\mu g/m^3$ (+5.9%). Its mean relative error is 18.36%, compared with 17.00 ± 0.67% in the paper. The local controls use the same withheld targets, but their preprocessing and implementation are independent of the paper's baseline code.

The local run did not reach the published mean MAE. It reduced MAE by 12.4% relative to the local interpolation control. Its MSE was lower than the published GRIN mean, showing that the ranking also depends on the error metric; this single run does not establish an overall improvement over the paper.

The reduced budget is the main planned difference: 480 local updates were allowed, compared with up to 48,000 for each paper-budget seed. One seed cannot establish variability comparable with five published runs. The random seeds from the original experiments were not supplied. Modern software versions, floating-point arithmetic and the shorter learning-rate schedule may also contribute to differences. Their individual effects were not isolated, so the observed difference cannot be attributed to a single cause.

The notebook verifies that training, validation and testing use disjoint timestamps; withheld validation and test targets never enter model inputs; and observed measurements remain unchanged after imputation. Additional tests check the five-term loss, model output shapes, invariance to hidden target values and evaluation coverage. Data, source and output checksums support rerunning and verifying the saved results.

Reproducing the experiment

The public repository includes the executed notebook, installation commands, pinned dependencies, preprocessing and training code, full parameter settings, a trained checkpoint, training history, predictions and measured metrics. Data and original model sources are downloaded from versioned locations with checksum verification. The README explains cached review, fresh training and the full-budget option.

https://github.com/Achiee7/dw-grin-reproduction

Notebook: notebooks/grin_aqi36_reproduction.ipynb. Run from the repository root using Python 3.12: python scripts/run_notebook.py. Add --retrain for a fresh local run; --profile paper --seed 42 selects one paper-budget run. Five separate seeds are required for a five-run comparison.

[1] Cini, A., Marisca, I. and Alippi, C. (2022). Filling the G_ap_s: Multivariate Time Series Imputation by Graph Neural Networks. ICLR. https://arxiv.org/abs/2108.00298v3.